SCHEDULO.AI - 1ST ASSESSMENT

PROJECT 01

Personal Task Manager

Tech Stack: Python · Django · SQLite · Bootstrap 5

Introduction

A Personal Task Manager is a web application that allows a user to create, organize, and track their daily tasks. Think of it as a simple, personal to-do list application with user accounts.

Backend Requirements

Django Models

The following data models need to be implemented:

- User — handled by Django's built-in auth system
- Task — the main model for storing task information
- Category — to group tasks (e.g., Work, Personal, Study)

FOR TASK MODEL - FIELDS:

Field	Description
title	Short name of the task (CharField)
description	Optional longer details (TextField, blank=True)
due_date	Deadline for the task (DateField)
priority	Low / Medium / High (CharField with choices)
status	Pending / In Progress / Done (CharField with choices)
category	ForeignKey to Category model
owner	ForeignKey to User (each task belongs to one user)
created_at	Auto-set timestamp when task is created

Views & URLs

Students should implement the following views using Django's function-based or class-based views:

- Home / Dashboard — show all tasks of the logged-in user
- Task Detail — view a single task's full details
- Add Task — form to create a new task
- Edit Task — pre-filled form to update an existing task

- Delete Task — confirm and delete a task
- Category List — view and manage categories
- Register / Login / Logout — user authentication views

Authentication & Permissions

- Users must be logged in to access any task-related pages
- Use Django's @login_required decorator on protected views
- A user should never be able to view or edit another user's tasks

Forms

- Use Django ModelForms for task creation and editing
- Add basic server-side validation (e.g., due date cannot be in the past)
- Show meaningful error messages on the form if validation fails

Frontend Requirements

The frontend should use Django's templating engine with Bootstrap 5 for styling.

Pages to Build

- Login & Registration Page — clean form, link between the two pages
- Dashboard / Task List — display all tasks in a table or card layout
- Task Form Page — shared template for both add and edit actions
- Task Detail Page — show all fields, with Edit and Delete buttons
- Category Management Page — list, add, and delete categories

UI/UX Expectations

- Navigation bar with links to Dashboard, Add Task, Categories, Logout
- Visual indicators for task priority (e.g., colored badges: Red = High, Yellow = Medium, Green = Low)
- Visual indicator for task status (e.g., strikethrough text for Done tasks)
- Flash messages using Django's messages framework (e.g., 'Task created successfully!')
- Responsive layout that works on both desktop and mobile screens

Expected Features Summary

Feature	Expected Behavior
User Registration	New user can sign up with username, email, password
User Login / Logout	Secure session-based login and logout
Create Task	Form with all fields; saved to DB linked to logged-in user

List Tasks	Dashboard shows all tasks; can filter by status or category
Edit Task	Pre-filled form; updates existing record in DB
Delete Task	Confirmation prompt before deletion
Task Priority Badges	Color-coded labels for Low, Medium, High
Category Filter	User can click a category to see only those tasks
Flash Notifications	Success/error messages after any action
Access Control	Logged-out users redirected to login page

PROJECT 02

Student Blog Platform

Tech Stack: Python · Django · SQLite · Bootstrap 5

Introduction

A Student Blog Platform is a multi-user blogging website where registered users can write, publish, and manage blog posts. Other users (and visitors) can read these posts and leave comments

Backend Requirements

Django Models

Model	Purpose
User (built-in)	Django's default authentication user
UserProfile	Extends User with a bio and profile picture
Post	The main blog post with title, body, author, tags
Comment	A comment left on a specific post by a user
Tag	A label/keyword that can be attached to posts

Post model fields:

- title — title of the blog post
- slug — URL-friendly version of the title (auto-generated)
- body — main content (TextField)
- author — ForeignKey to User
- created_at and updated_at — auto timestamps

- status — Draft or Published
- thumbnail — optional image upload for the post cover
- tags — ManyToManyField to Tag model

Comment model fields:

- post — ForeignKey to Post
- author — ForeignKey to User
- body — the comment text
- created_at — timestamp

Views & URLs

- Home / Blog Feed — list of all published posts (newest first)
- Post Detail — full post content + comments section
- Create Post — form to write and publish a new post
- Edit Post — update an existing post (only the author can do this)
- Delete Post — only the author can delete their post
- My Posts — author's own posts page (includes drafts)
- User Profile Page — public page showing a user's published posts
- Tag Filter — click a tag to see all posts with that tag
- Register / Login / Logout

Business Logic & Permissions

- Only logged-in users can write posts or leave comments
- Drafts are only visible to the author, not to the public
- Only the post author can edit or delete their own posts (use get_object_or_404 with author check)
- Comment submission should be done via a form at the bottom of the post detail page
- Slug should be auto-generated from the title using Django's slugify utility

File Upload

- Configure MEDIA_URL and MEDIA_ROOT in settings.py
- Store uploaded thumbnails in a media/thumbnails/ folder
- Show a default placeholder image if no thumbnail is uploaded

Frontend Requirements

Pages to Build

- Home / Feed — blog card grid showing post thumbnail, title, author, date, and tags
- Post Detail Page — full content, author info, comment list, and add-comment form
- Post Form Page — rich textarea form for creating and editing posts
- My Posts Page — table of the author's own posts with status badge and actions

- User Profile Page — avatar, bio, and list of the user's published posts
- Login & Register Pages

UI/UX Expectations

- Navigation bar with logo, Home, Write Post, My Posts, Profile, Logout
- Blog cards on the home page with a thumbnail, title, short excerpt, author name, and date
- Tag badges on cards and post detail page; clickable to filter posts
- Pagination on the home feed (show 6 posts per page using Django's Paginator)
- Comment section below the post with a form for logged-in users to post comments
- Draft indicator on the author's 'My Posts' page
- Flash messages for post created, updated, deleted, and comment posted

Expected Features Summary

Feature	Expected Behavior
User Registration & Login	Secure sign-up and sign-in with session management
Create Post	Form with title, body, tags, thumbnail upload, and status toggle
Draft vs Published	Drafts visible only to the author; published visible to all
Post Detail Page	Full content, author, date, tags, and comment section
Leave a Comment	Logged-in users can comment on any published post
Edit / Delete Post	Only the post author can modify or remove their post
Tag Filtering	Clicking a tag filters the home feed to only matching posts
Pagination	Home feed shows 6 posts per page with Previous/Next controls
User Profile	Public page showing user bio and their published posts
Thumbnail Upload	Optional post cover image; placeholder shown if none uploaded

Good luck and happy coding!

Reach out to us if you get stuck , asking for help is part of learning :)